Лабораторна робота №1

Мета роботи: Сформувати розуміння архітектури ROS2, принципів роботи нод, топіків, механізму Publisher/Subscriber та навчитися створювати власний ROS2-пакет і реалізовувати обмін повідомленнями між нодами.

Обладнання та програмне забезпечення:

- ПК з ОС Windows, віртуальна машина VirtualBox з Ubuntu 22.04 LTS;
- ROS2 Humble Hawksbill (ros-humble-desktop);
- система збірки colcon;
- IDE Visual Studio Code;
- мова програмування Python 3.10, клієнтська бібліотека rclpy.

Хід роботи

1. Перевірка встановлення ROS2 та запуск прикладу turtlesim

Для перевірки коректності встановлення ROS2 було виведено змінну середовища ROS_DISTRO (значення humble) та запущено демонстраційні ноди talker (пакет demo_nodes_cpp) і listener (пакет demo_nodes_py). Нода talker публікувала повідомлення Hello World: N, а listener їх успішно отримувала, що підтверджує працездатність ROS2 та проміжного шару DDS (рис. 1).

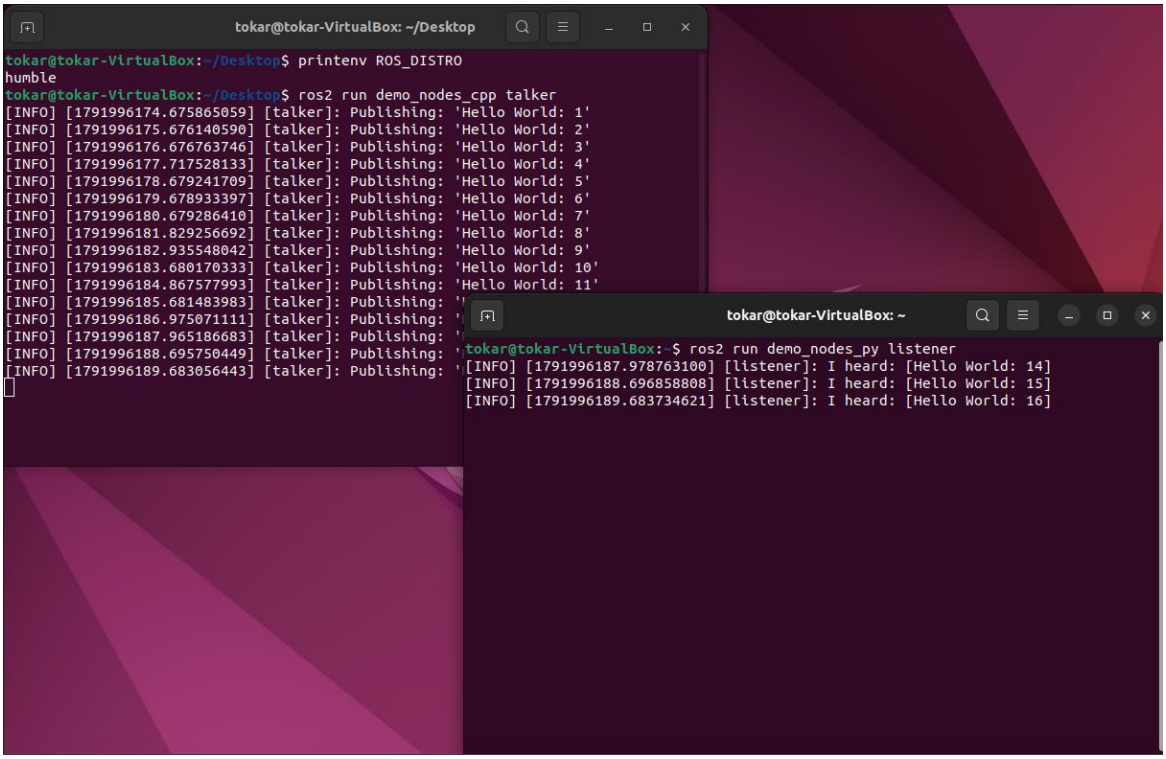


Рисунок 1

Далі було запущено симулятор turtlesim командою ros2 run turtlesim turtlesim_node, у результаті чого відкрилося графічне вікно з черепахою. В

окремому терміналі запущено ноду керування ros2 run turtlesim turtle_teleop_key, яка зчитує натискання клавіш-стрілок і перетворює їх на команди руху. Керуючи черепахою з клавіатури, було отримано траєкторію руху (рис. 2).

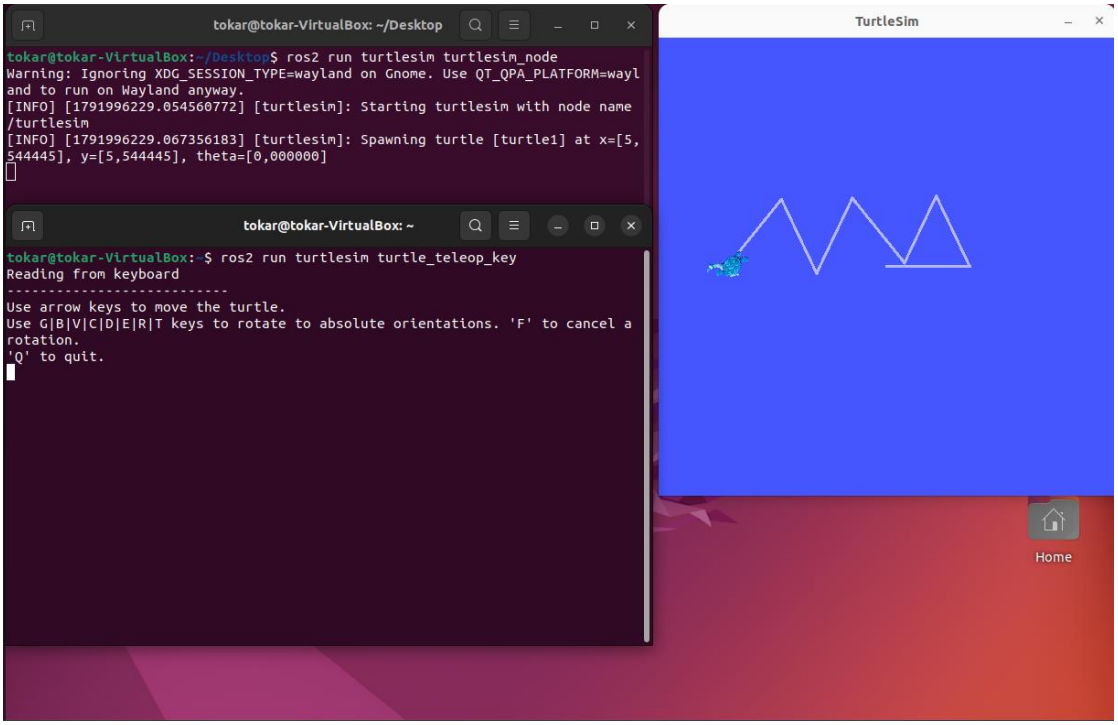


Рисунок 2

За допомогою команди ros2 node list визначено активні ноди системи (рис. 3).

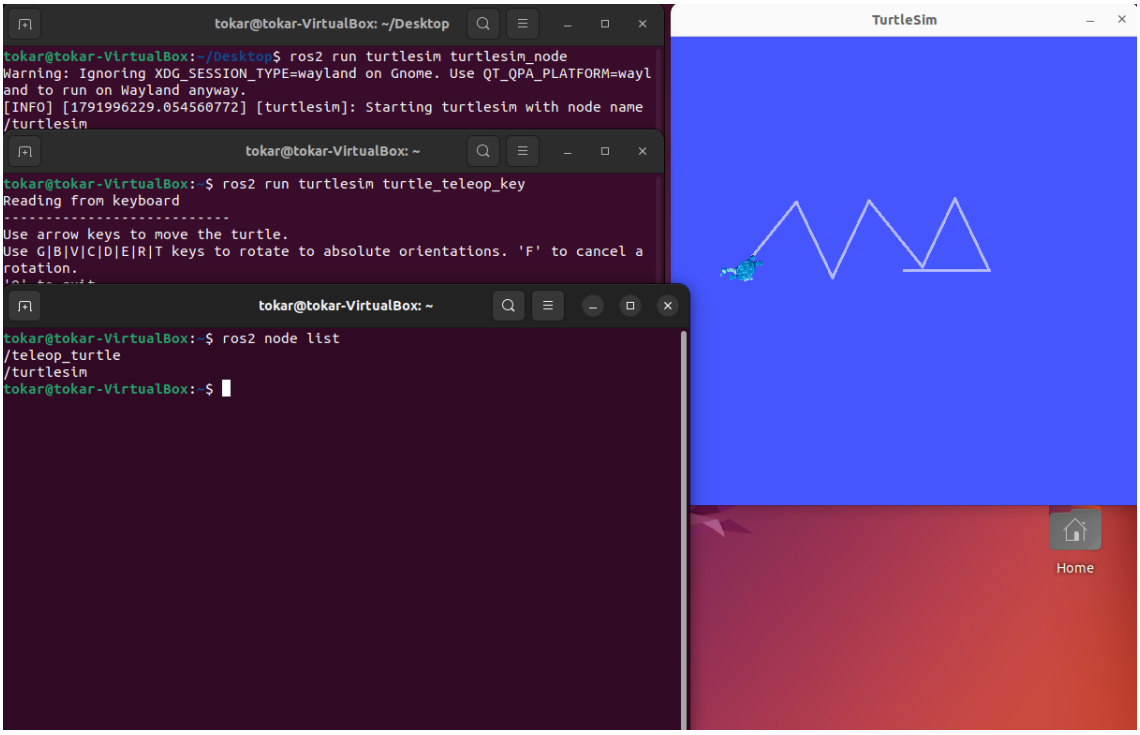


Рисунок 3

Командою ros2 topic list -t отримано перелік активних топіків та типи їх повідомлень (рис. 4).

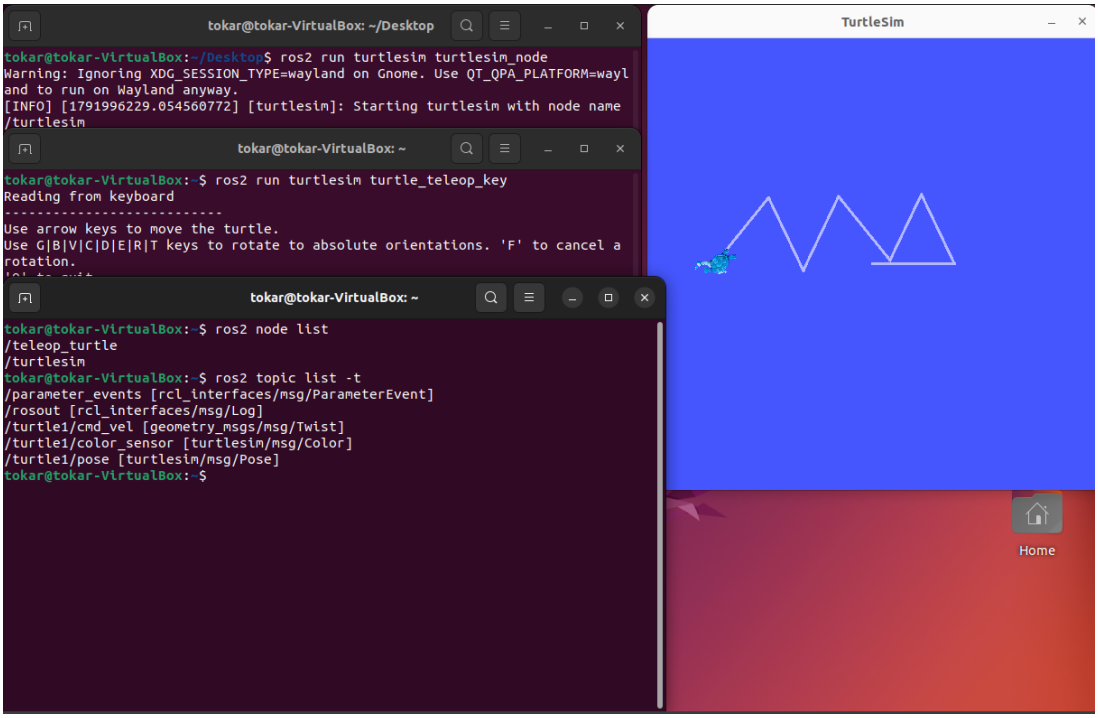


Рисунок 4

Для перегляду повідомлень було виконано команду ros2 topic echo /turtle1/cmd_vel. Під час натискання клавіш у терміналі у консоль виводилися повідомлення, що містять лінійну та кутову швидкості (рис. 5). Це демонструє, що нода /teleop_turtle є Publisher, а /turtlesim є Subscriber топіка /turtle1/cmd_vel.

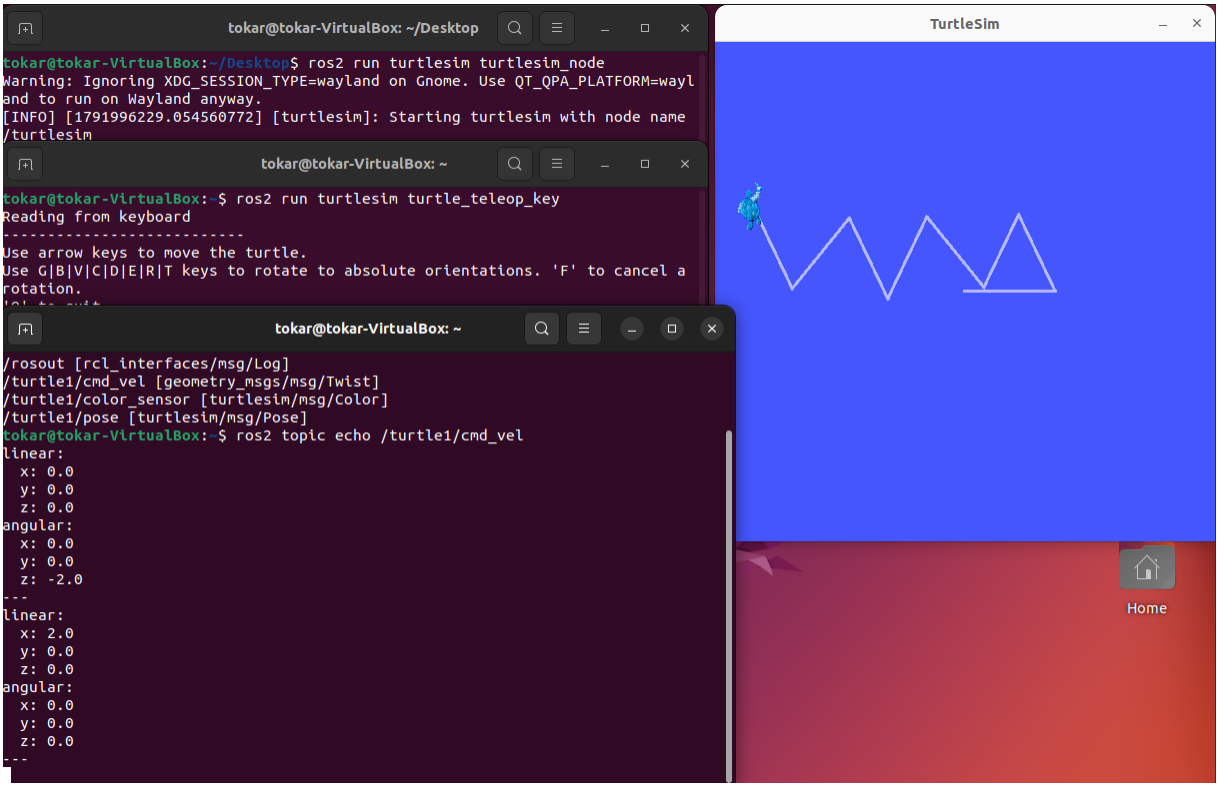


Рисунок 5

2. Створення власного пакету

Створено робочий простір ~/ros2_ws з каталогом src, у якому командою ros2 pkg create --build-type ament_python --license Apache-2.0 --dependencies rclpy std_msgs --node-name publisher_node lab1_pubsub створено пакет lab1_pubsub з типом збірки ament_python та залежностями rclpy і std_msgs (рис. 6).

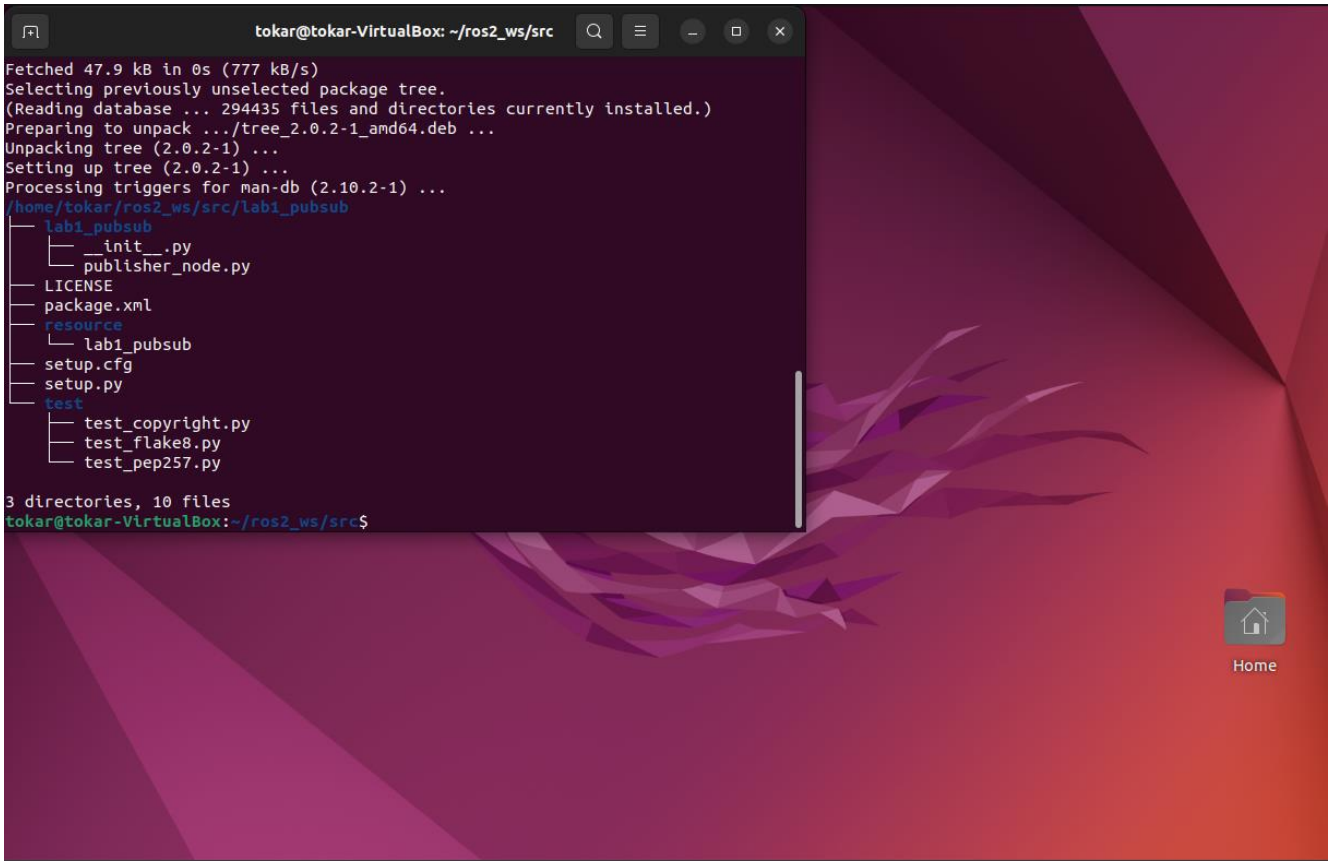


Рисунок 6

Робочий простір було зібрано командою colcon build, після чого середовище активовано командою source install/setup.bash. Коректність встановлення пакету перевірено командами ros2 pkg list, ros2 pkg prefix lab1_pubsub та ros2 pkg executables lab1_pubsub (рис. 7).

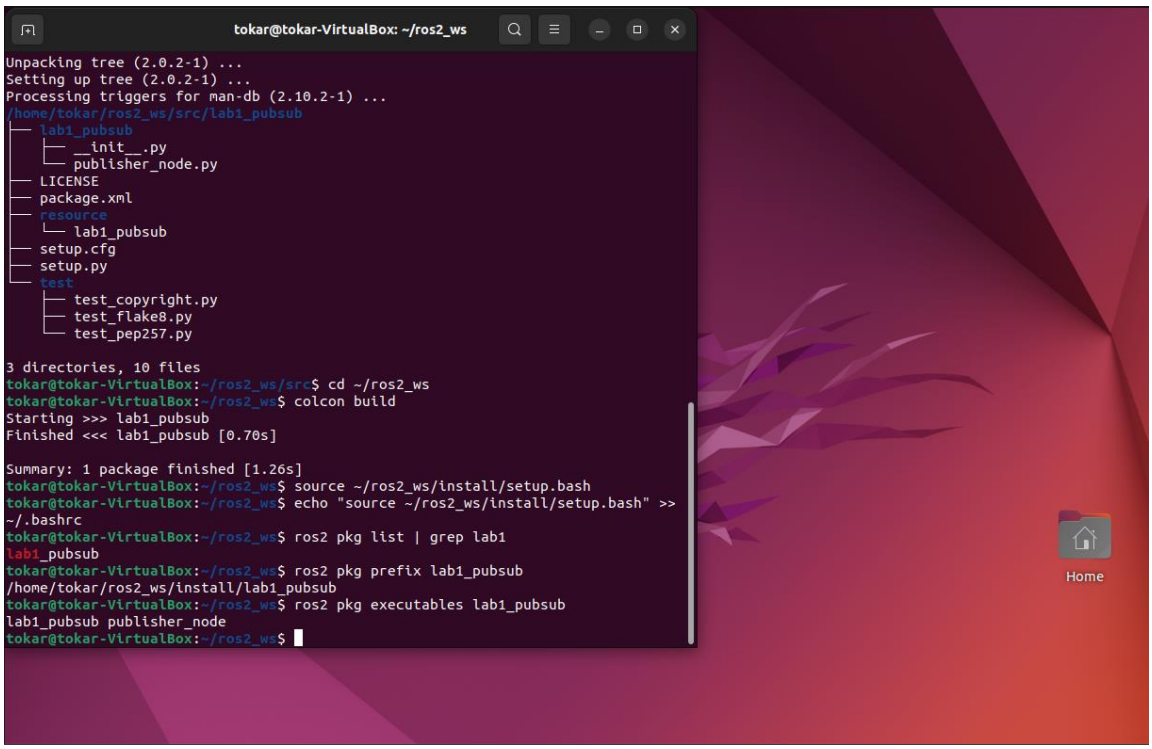


Рисунок 7

3. Реалізація Publisher

Нода-публікатор lab1_publisher реалізована у вигляді класу Lab1Publisher, успадкованого від rclpy.node.Node. У конструкторі створюється Publisher для топіка lab1_topic з типом повідомлення std_msgs/msg/String та глибиною черги 10, а також таймер з періодом 1,0 с. У функції зворотного виклику таймера формується текстове повідомлення з порядковим номером, воно публікується у топік, а факт публікації фіксується в журналі на рівні INFO.

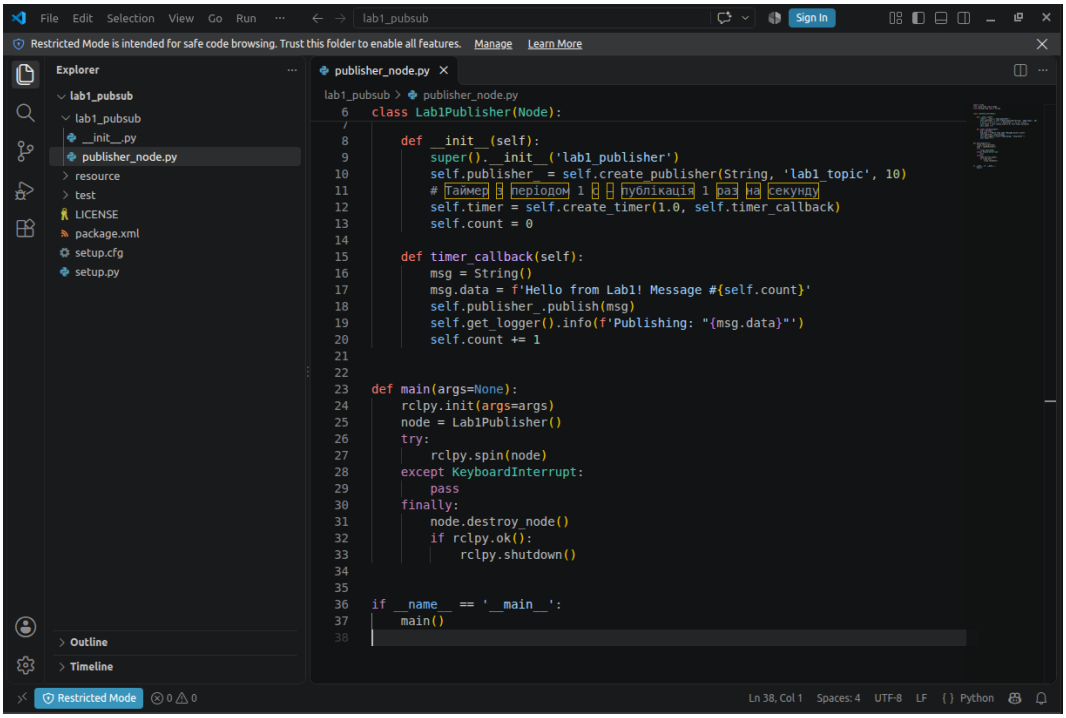


Рисунок 8

4. Реалізація Subscriber

Нода-підписник lab1_subscriber реалізована класом Lab1Subscriber. У конструкторі створюється підписка на топік lab1_topic того ж типу std_msgs/msg/String. При надходженні кожного повідомлення викликається функція listener_callback, яка виводить отриманий текст у консоль через логер ROS2.

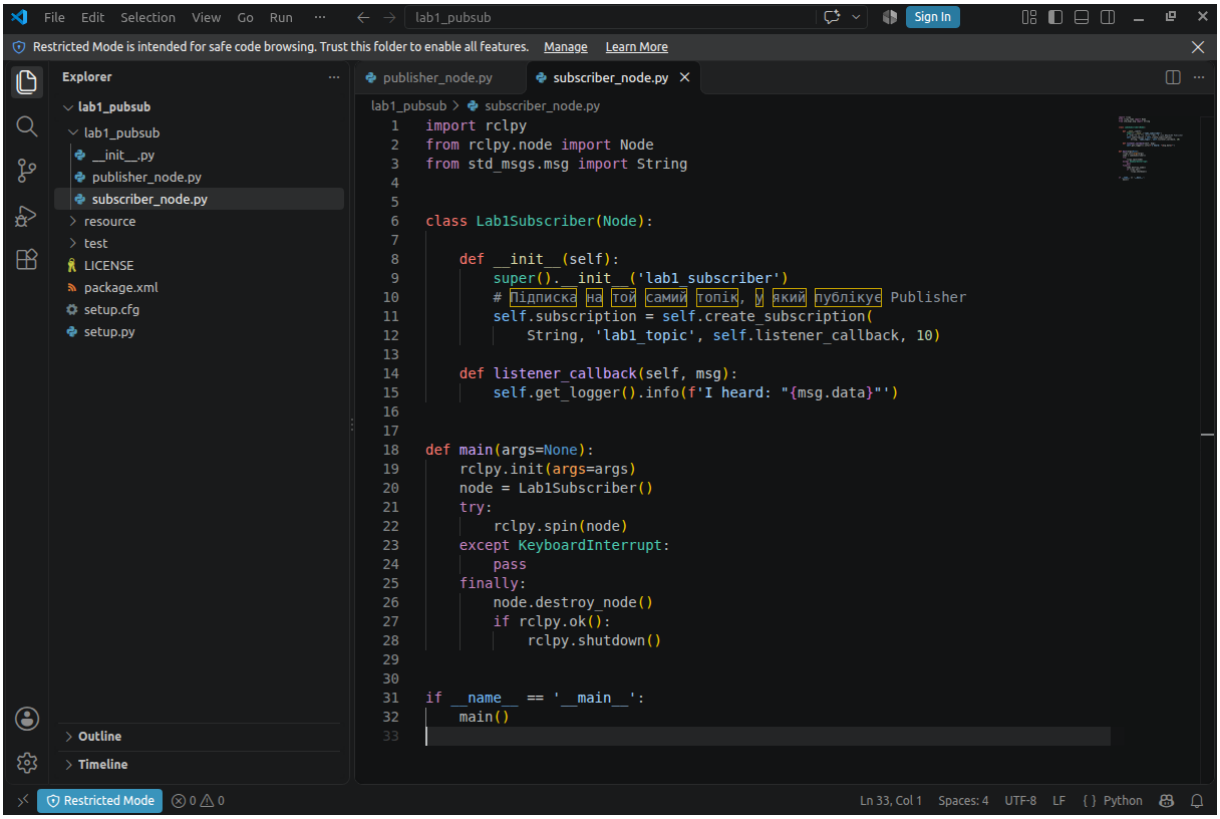


Рисунок 9

Для того щоб ноди можна було запускати командою ros2 run, їх зареєстровано у файлі setup.py (рис. 10).

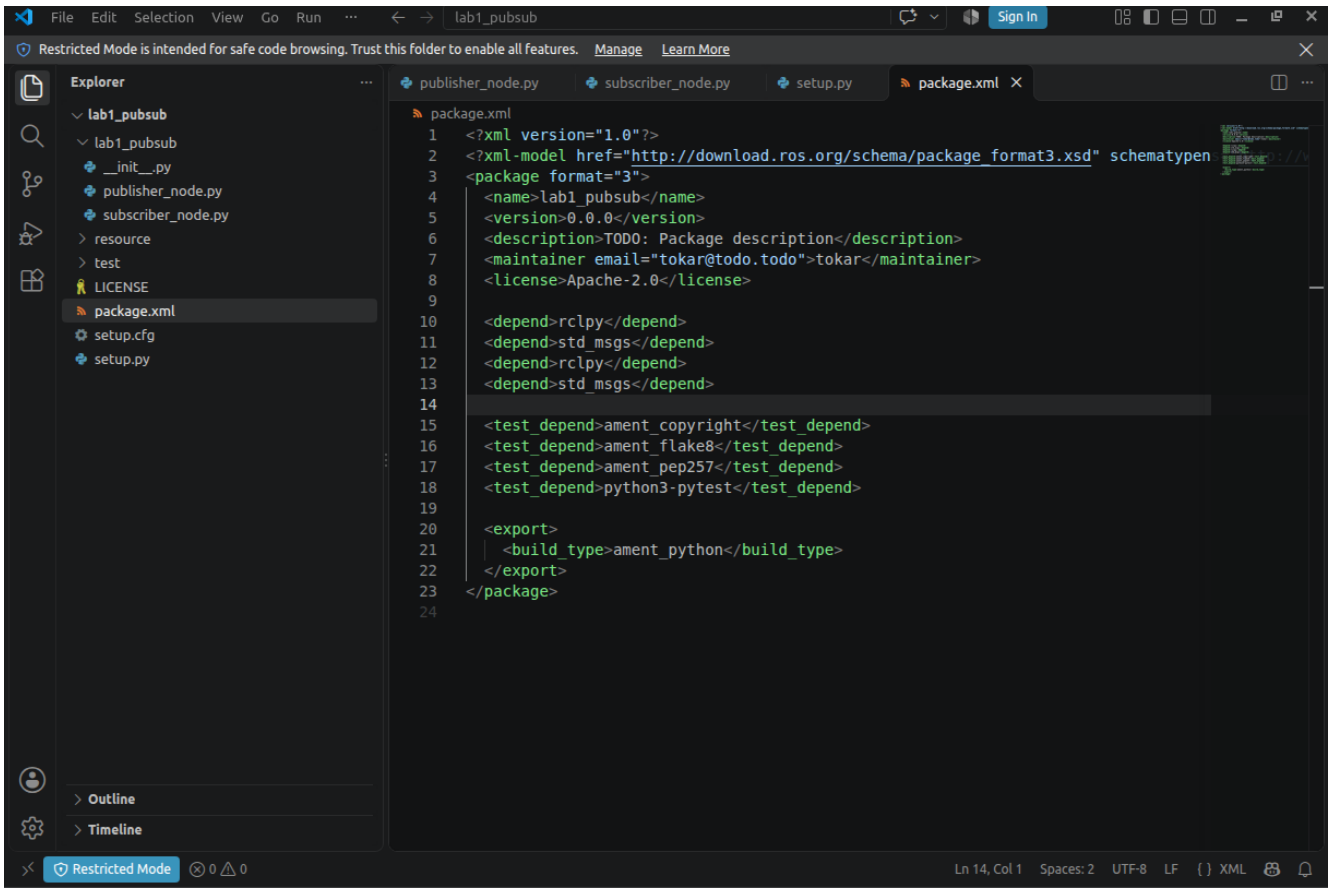


Рисунок 10

5. Перевірка роботи системи

Publisher і Subscriber запущено в окремих терміналах командами ros2 run lab1_pubsub publisher_node та ros2 run lab1_pubsub subscriber_node. Publisher щосекунди публікує повідомлення та виводить "Hello from Lab1! Message #N", а Subscriber отримує ці повідомлення та виводить I heard: "Hello from Lab1! Message #N". Номери повідомлень збігаються, що свідчить про коректний обмін даними (рис. 11).

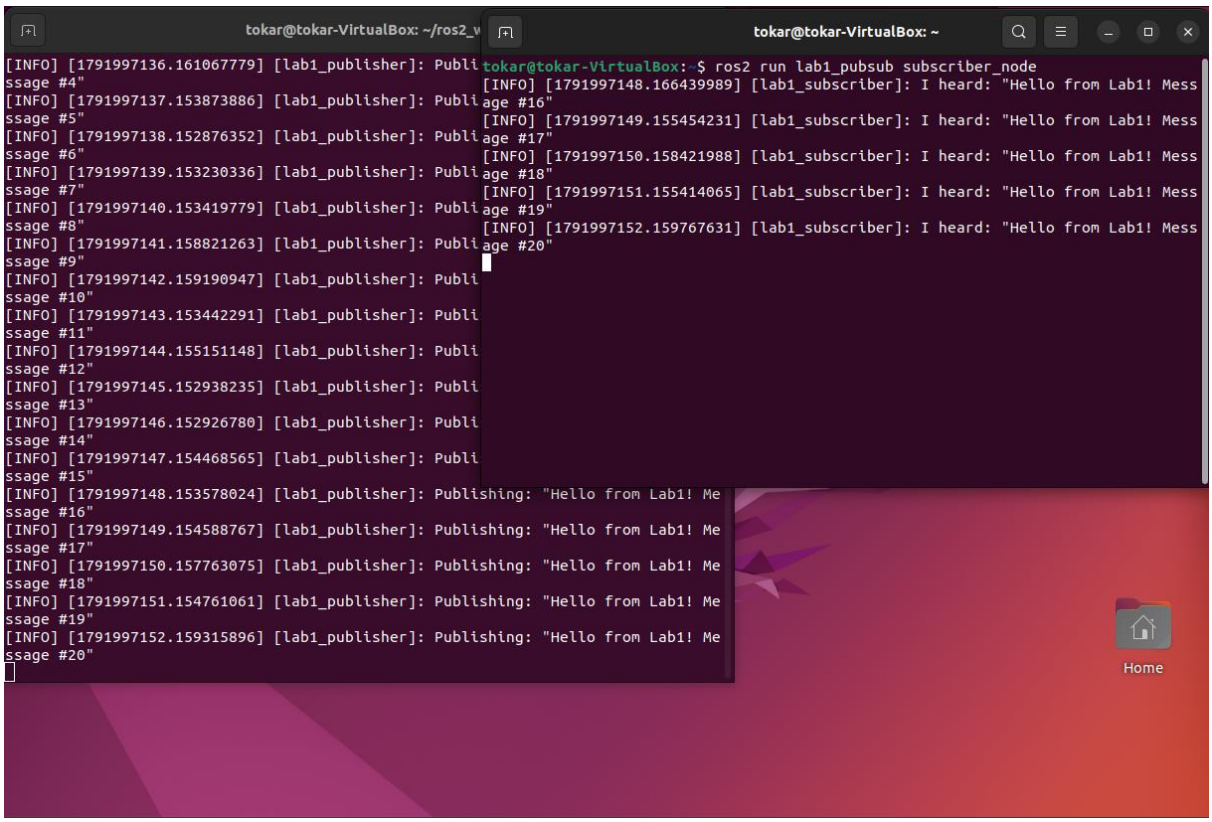


Рисунок 11

Відображення нод і топіків перевірено стандартними командами ROS2 (рис. 12).

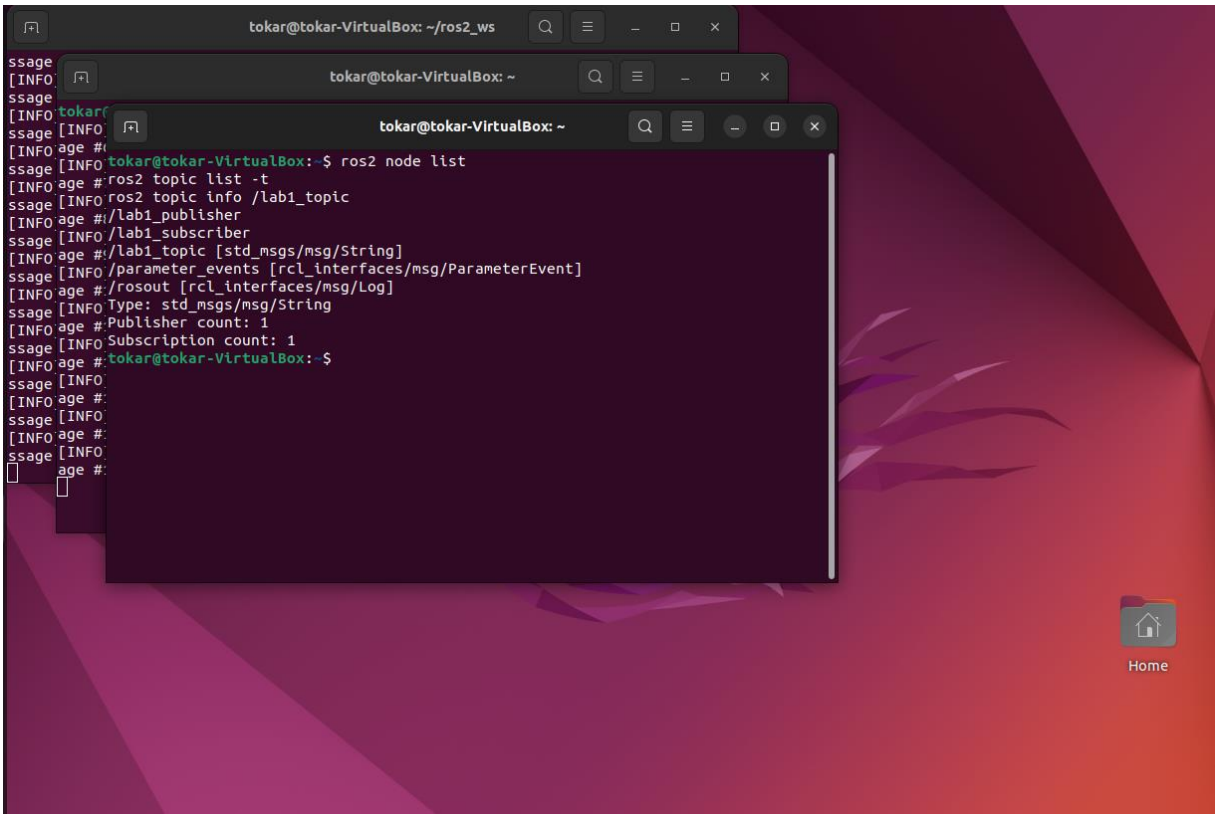


Рисунок 12

Команда ros2 topic echo /lab1_topic відобразила повідомлення, що передаються через топік, а ros2 topic hz /lab1_topic підтвердила частоту публікації +-1 Гц, що відповідає вимозі завдання (рис. 13).

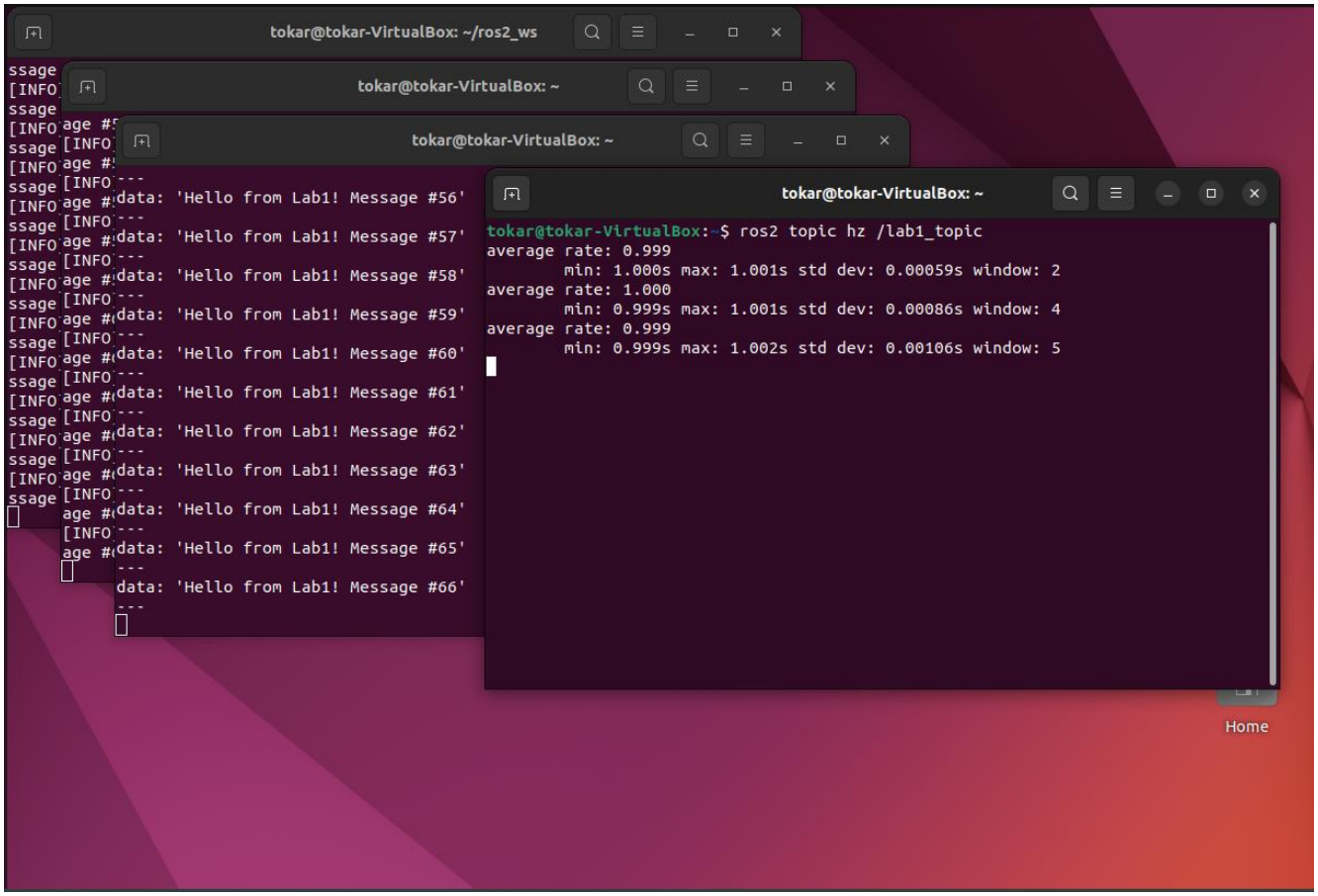


Рисунок 13

Висновок до лабораторної роботи:

Під час лабораторної роботи було розглянуто архітектуру ROS2 та її модульний принцип: система складається з окремих вузлів (нод), які обмінюються даними через іменовані топіки за моделлю «публікатор–підписник» і не мають прямого зв’язку один з одним. На прикладі turtlesim було показано взаємодію двох нод: /teleop_turtle надсилає команди швидкості в топік /turtle1/cmd_vel, а /turtlesim підписується на нього та рухає черепаху. За допомогою команд ros2 node list, ros2 topic list і ros2 topic echo було визначено активні ноди й топіки, а також переглянуто вміст повідомлень типу geometry_msgs/msg/Twist. Було створено власний пакет lab1_pubsub із типом збірки ament_python. У ньому реалізовано ноду-публікатор, яка щосекунди надсилає повідомлення типу std_msgs/msg/String із логуванням на рівні INFO, та ноду-підписник, яка отримує ці повідомлення й виводить їх у консоль через логер ROS2. Коректність обміну підтверджено збігом номерів повідомлень, виводом стандартних команд ROS2 та вимірюванням частоти публікації — приблизно 1 Гц.